

CDC GSIPG : LA PETITE GLORIA

Table des matières

1. Compréhension du contexte et du problème	3
1.1 Présentation du contexte.....	3
1.2 Identification du problème réel	3
1.3 Analyse de l'existant (AS-IS)	3
Le site actuel présente des informations textuelles denses sans hiérarchie claire ni parcours utilisateur incitatif (absence de tunnels de conversion/d'inscription fluides).....	
2. Définition de la vision de la solution	3
2.1 Définition des objectifs	3
Objectifs Techniques : Atteindre un score de performance Google Lighthouse > 90/100, un temps de chargement initial (First Contentful Paint) inférieur à 1,5 seconde, et un responsive design parfait.	
2.2 Définition de la proposition de valeur.....	3
Offrir une expérience web immersive, moderne et ultra-rapide qui prouve graphiquement le standing international de l'établissement.....	
3. Étude de faisabilité	3
Pour garantir le site le plus léger et fluide possible, la stack technique retenue élimine les architectures lourdes (type WordPress ou gros frameworks monolithiques) au profit de la Jamstack :	
• Framework : Astro ou Next.js (en mode Static Site Generation - SSG). Astro est idéal ici car il génère du HTML pur sans JavaScript par défaut, garantissant une légèreté absolue.	
3	
• Styling : Tailwind CSS (compilé et purgé pour ne conserver que le CSS réellement utilisé, soit un fichier final de quelques dizaines de Ko seulement).	
3	
• Gestion des médias (Vidéos/Images) : Hébergement des vidéos du parc sur un CDN optimisé (Cloudflare Stream ou Vimeo) avec chargement différé (<i>lazy-loading</i>) pour ne pas bloquer le rendu du texte et du design.....	
3	
4. Recueil des besoins (Spécifique Frontend)	4
4.2 Besoins non fonctionnels (Performance & Sécurité)	4
5. Analyse fonctionnelle.....	4

5.1 Identification des acteurs.....	4
Parents d'élèves (principaux décideurs), élèves (co-décideurs via l'attrait des parcs), administration de l'école (réceptionnaire des leads).	
6.2 Cas d'utilisation.....	4
5.2 Parcours utilisateurs	4
6. Conception de l'architecture et DevOps	4
6.1 Architecture applicative.....	4
Site entièrement pré-rendu (SSG) déployé sur un réseau de serveurs mondiaux (CDN Vercel ou Netlify). Zéro base de données connectée directement au frontend pour éviter les failles de sécurité et les lenteurs de requêtes.....	4
6.2 Pipeline DevOps (CI/CD)	5

1. Compréhension du contexte et du problème

1.1 Présentation du contexte

Le GSIPG modernise son infrastructure d'accueil en intégrant de nouveaux parcs d'attractions. Le site web actuel (gsipg.com) doit être refondu pour refléter cette double identité (excellence académique et loisirs éducatifs).

1.2 Identification du problème réel

L'ancien site souffre d'une interface vieillissante, d'un manque d'attractivité visuelle pour les parents modernes, et de performances d'affichage qui ralentissent l'accès sur les connexions mobiles (critique pour la cible locale au Cameroun).

1.3 Analyse de l'existant (AS-IS)

Le site actuel présente des informations textuelles denses sans hiérarchie claire ni parcours utilisateur incitatif (absence de tunnels de conversion/d'inscription fluides).

2. Définition de la vision de la solution

2.1 Définition des objectifs

Objectifs Métier : Capter l'attention des parents en moins de 3 secondes, valoriser le concept ludo-pédagogique unique, et générer des demandes d'inscription qualifiées.

Objectifs Techniques : Atteindre un score de performance Google Lighthouse > 90/100, un temps de chargement initial (First Contentful Paint) inférieur à 1,5 seconde, et un responsive design parfait.

2.2 Définition de la proposition de valeur

Offrir une expérience web immersive, moderne et ultra-rapide qui prouve graphiquement le standing international de l'établissement.

3. Étude de faisabilité

Pour garantir le site le plus léger et fluide possible, la stack technique retenue élimine les architectures lourdes (type WordPress ou gros frameworks monolithiques) au profit de la Jamstack :

- **Framework :** Astro ou Next.js (en mode Static Site Generation - SSG). Astro est idéal ici car il génère du HTML pur sans JavaScript par défaut, garantissant une légèreté absolue.
- **Styling :** Tailwind CSS (compilé et purgé pour ne conserver que le CSS réellement utilisé, soit un fichier final de quelques dizaines de Ko seulement).
- **Gestion des médias (Vidéos/Images) :** Hébergement des vidéos du parc sur un CDN optimisé (Cloudflare Stream ou Vimeo) avec chargement différé (*lazy-loading*) pour ne pas bloquer le rendu du texte et du design.

4. Recueil des besoins (Spécifique Frontend)

- Lecture fluide de vidéos natives d'enfants en train de jouer et d'étudier sans ralentissement du site.
- Formulaire d'inscription interactif léger en 4-5 champs avec validation instantanée côté client.
- Menu de navigation collant (*sticky*) et interactif pour un accès permanent aux contacts.
- Intégration d'une carte géographique allégée (Mapbox Lite ou sur-mesure) pour situer l'école.

4.2 Besoins non fonctionnels (Performance & Sécurité)

- Performance : Optimisation agressive des images au format nouvelle génération (.webp / .avif).
- Accessibilité : Fluidité de navigation même en mode de connexion dégradée (3G/4G).
- Sécurité : Protection des formulaires contre le spam (Cloudflare Turnstile ou Honeypot discret, plus légers que ReCAPTCHA).

5. Analyse fonctionnelle

5.1 Identification des acteurs

Parents d'élèves (principaux décideurs), élèves (co-décideurs via l'attrait des parcs), administration de l'école (réceptionnaire des leads).

6.2 Cas d'utilisation

5.2 Parcours utilisateurs

- Le parent arrive sur le site → Captivé par la vidéo du parc et de la classe (Hook).
- Fait défiler la page → Découvre les atouts pédagogiques sous forme de cartes fluides (What).
- Comprend comment s'y prendre → Visualise la timeline claire en 3 étapes (How).
- Passe à l'action → Remplit le mini-formulaire ou clique sur le bouton d'appel direct (CTA).

6. Conception de l'architecture et DevOps

6.1 Architecture applicative

Site entièrement pré-rendu (SSG) déployé sur un réseau de serveurs mondiaux (CDN Vercel ou Netlify). Zéro base de données connectée directement au frontend pour éviter les failles de sécurité et les lenteurs de requêtes.

6.2 Pipeline DevOps (CI/CD)

Déploiement automatisé à chaque modification du code sur GitHub/GitLab. Tests de performance automatisés à chaque build pour vérifier qu'aucune image lourde n'altère la vitesse du site.